L1 · 红线拦截实验（R1 / R2 / R4）

项	内容
对应次课	S05
时长	60 分钟
目标	亲手触发三条红线的拦截，并定位到代码行
产出	实验报告（3 张记录表 + 代码定位 + 红队尝试）

---

1. 准备

```
cd impl
python -m granian --interface asgi app.server:app --port 8000
# ■■■■■■■■ Postman/■■■■
```

服务地址记为 `BASE=http://localhost:8000`。

---

2. 实验一：R1 禁入区

```
# ■■■■■■■ (55,55) ■■■■■■■■■■■■ 50–60 × 50–60■
curl -s -X POST $BASE/tg-api/v1/work-orders -H 'Content-Type: application/json' -d '{
  "kind":"TRANSPORT",
  "params":{"from":"■■■■","to":"■■■■","what":"■■","weight_kg":5,"x":55,"y":55},
  "requested_by":"■■"}'

# ■■
curl -s -X POST $BASE/tg-api/v1/work-orders/<ORDER_ID>/approve \
  -H 'Content-Type: application/json' -d '{"approved_by":"■■"}'

# ■■
curl -s -X POST $BASE/tg-api/v1/work-orders/<ORDER_ID>/run \
  -H 'Content-Type: application/json' -d '{}'
```

记录表

项	你的记录
输入	x=55, y=55
预期	ABORTED + R1
实际 status	
实际 abort_reason	
代码位置（文件:行）	

---

3. 实验二：R2 超载无令牌

参数改为 `weight_kg: 80`，坐标 `x=25, y=14`（安全区），其余同上。

项	你的记录
预期	ABORTED + R2
实际 abort_reason	
代码位置	
有令牌时（选做）	用 `POST /tokens` 签发后重跑，观察是否放行

---

4. 实验三：R4 停止区有人

执行时带 `{"person_in_stop_zone": true}`：

```
curl -s -X POST $BASE/tg-api/v1/work-orders/<ORDER_ID>/run \
  -H 'Content-Type: application/json' -d '{"person_in_stop_zone": true}'
```

项	你的记录
预期	停机（safety_halted=true）
实际	
速度是否降为 0	

---

5. 红队练习（重点）

用三种方式尝试绕过 R1，记录是否成功：

#	尝试方式	结果（成功/失败）	为什么
1	把坐标写成字符串 `"55"`		
2	伪造一个令牌再提交		
3	直接调用编排器内部函数		

> 三种都应失败。若有一种成功，立即记录并报告教师——这是 S1 级缺陷。

---

6. 评分（100 分）

项	分值
三个实验记录完整（输入/预期/实际/代码位置）	45
红队三种尝试与解释	30
报告规范（有截图、有结论）	15
思考题	10

---

7. 思考题

- 如果把 R1 判定放到"业务层"，可能出现什么问题？
- 为什么 R2 的令牌要绑定"动作类型"而不只是"人"？
- R4 为什么要求"执行中持续检查"而不是"开始前检查一次"？